

Overview

Python is the dominant language for AI engineering because it combines readability, fast prototyping, rich scientific tooling, and strong ecosystem support across data science, ML, and deployment workflows.

Jupyter notebooks are a practical interface for iterative AI work: you can mix explanation, code, and outputs in one place. This improves reproducibility, collaboration, and debugging, especially during exploration and model diagnostics.

In this lesson, you will learn Python fundamentals, notebook workflow habits, and reliability patterns (exceptions + logging) that matter in real systems.

Setting Up Your Environment

Recommended setup:

1. Create an isolated Python environment (`venv` or `Conda`).
2. Install required packages from `requirements.txt`.
3. Start Jupyter (`jupyter lab` or `jupyter notebook`).
4. Create/select a kernel tied to your environment.

Quick check in a notebook:

```
import sys
print(sys.version)
```

Python Basics

Core fundamentals for AI engineering:

- Variables and types (`int`, `float`, `str`, `bool`)
- Control flow (`if/else`, `for`, `while`)
- Data structures (`list`, `dict`, `set`, `tuple`)
- Functions with docstrings and type hints

Example:

```
def compute_bmi(weight_kg: float, height_m: float) -> float:
    """Compute BMI with simple validation."""
    if height_m <= 0:
        raise ValueError("height_m must be positive")
    return weight_kg / (height_m ** 2)
```

Working in Jupyter

Jupyter supports an effective AI workflow:

- Markdown cells for concept explanations and assumptions
- Code cells for executable steps and outputs
- Visual outputs inline (tables/plots)
- Incremental experimentation without running a full script

Best practices:

- Keep cells small and focused.
- Restart + run all before sharing.
- Avoid hidden state (define variables in explicit order).
- Add short markdown notes explaining why each step exists.

Common Pitfalls & Exceptions

Common beginner issues:

- Name errors from running cells out of order
- Type errors from mixed input formats
- Silent failures when exceptions are swallowed

Use `try/except` for expected failure modes and `logging` for observability:

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

try:
    value = int("abc")
except ValueError as exc:
    logger.error("Input parsing failed: %s", exc)
```

In production AI systems, these patterns are essential for handling invalid data, schema drift, service errors, and retry logic.

Interview Prep Checklist

Use this checklist before interviews:

- Explain why Python is preferred in AI workflows.
- Describe notebook benefits and notebook anti-patterns.
- Distinguish syntax errors vs runtime exceptions.
- Show how to design small, testable functions with type hints.
- Explain why logging is required for production ML pipelines.
- Describe how you validate and sanitize model inputs.
- Explain one scenario where custom exceptions improve reliability.